
Big Data & Data Analytics

Universidad Iberoamericana
Dr. Lázaro Bustio Martínez
lazaro.bustio@ibero.mx
Otoño de 2026

Tarea Sesión 4

Ejercicio de visualización en Python

En este ejercicio se estudiarán las principales librerías para la visualización de datos con Python y todos los tipos de gráficos que se pueden hacer con ellas. También se verán qué librería es recomendable utilizar en cada ocasión y las capacidades únicas de cada librería.

Datasets

En este ejercicio se trabajará con dos conjuntos de datos que se adaptarán a las visualizaciones que se mostrarán. Los conjuntos de datos se pueden descargar [aquí](#)¹. Los datasets describen datos sobre la popularidad de las búsquedas en Internet de tres términos relacionados con la inteligencia artificial (“ciencia de datos”, “aprendizaje automático” y “aprendizaje profundo”). Los datos para analizar se componen de dos archivos: “*temporal.csv*” y “*mapa.csv*”. “*temporal.csv*” incluye los datos de popularidad de los tres términos a lo largo del tiempo (de enero de 2004 a febrero de 2020). Además, se añadió una variable llamada “*categorical*” para demostrar la funcionalidad de los gráficos con variables categóricas. El archivo “*mapa.csv*” incluye los datos de popularidad separados por países. Se utilizará en la última sección del ejercicio para trabajar con visualizaciones de mapas.

Pandas

Antes de pasar a métodos más complejos, vamos a empezar con la forma más básica de visualizar los datos. Simplemente usaremos pandas para ver los datos y hacernos una idea de cómo están distribuidos.

Lo primero que debemos hacer es visualizar algunos ejemplos para ver qué columnas hay, qué información contienen, cómo están codificados los valores...

```
import pandas as pd
df = pd.read_csv('temporal.csv')
df.head(10) #View first 10 data rows
```

¹ <https://github.com/albertsl/datasets/tree/master/popularidad>

Copie el anterior fragmento de código y péguelo en Python, luego obtenga una captura de pantalla del resultado y péguelo aquí:

< ¡Resultado del fragmento de código aquí!>

Con el comando `describe` veremos cómo se distribuyen los datos, los máximos, los mínimos, la media, ...

```
df.describe()
```

Copie el anterior fragmento de código y péguelo en Python, luego obtenga una captura de pantalla del resultado y péguelo aquí:

< ¡Resultado del fragmento de código aquí!>

With the `info` command we will see what type of data each column includes. We could find the case of a column that when viewed with the `head` command seems numeric but if we look at subsequent data there are values in string format, then the variable will be coded as a string.

Con el comando `info` veremos qué tipo de datos incluye cada columna. Podríamos encontrar el caso de una columna que al verla con el comando `head` parece numérica, pero si miramos los datos posteriores hay valores en formato de cadena, entonces la variable se codificará como cadena.

```
df.info()
```

Copie el anterior fragmento de código y péguelo en Python, luego obtenga una captura de pantalla del resultado y péguelo aquí:

< ¡Resultado del fragmento de código aquí!>

Por defecto, Pandas limita el número de filas y columnas que muestra. Esto puede ser molesto porque se puede querer ver todos los datos.

c_32	c_33	c_34	c_35	...	c_460	c_461	c_462	c_463	c_464
1	1	1	1	...	1	1	1	1	1
1	1	1	1	...	1	1	1	1	1
1	1	1	1	...	1	1	1	1	1
1	1	1	1	...	1	1	1	1	1
1	1	1	1	...	1	1	1	1	1
...	...	...	...	...	...	...	...	...	...
1	1	1	1	...	1	1	1	1	1
1	1	1	1	...	1	1	1	1	1
1	1	1	1	...	1	1	1	1	1
1	1	1	1	...	1	1	1	1	1
1	1	1	1	...	1	1	1	1	1
1	1	1	1	...	1	1	1	1	1

Con estos comandos, aumentamos los límites, y podemos visualizar la totalidad de los datos. Cuidado con esta opción para conjuntos de datos grandes, podemos tener problemas para mostrarlos.

```
pd.set_option('display.max_rows', 500)
pd.set_option('display.max_columns', 500)
pd.set_option('display.width', 1000)
```

Copie el anterior fragmento de código y péguelo en Python, luego obtenga una captura de pantalla del resultado y péguelo aquí:

< ¡Resultado del fragmento de código aquí! >

Utilizando los estilos de Pandas, podemos obtener mucha más información al visualizar la tabla. En primer lugar, definimos un diccionario de formatos para que los números se muestren de forma legible (con un determinado número de decimales, fecha y hora en un formato relevante, con un porcentaje, con una moneda, ...) No te asustes, esto es sólo una visualización y no cambia los datos, no tendrás ningún problema para procesarlos después.

Para dar un ejemplo de cada tipo, he añadido los símbolos de moneda y porcentaje, aunque no tengan ningún sentido para estos datos.

```
format_dict = {'data science': '${0:,.2f}', 'Mes': '{:%m-%Y}', 'machine learning': '{:.2%}'}

#We make sure that the Month column has datetime format
df['Mes'] = pd.to_datetime(df['Mes'])

#We apply the style to the visualization
df.head().style.format(format_dict)
```

Copie el anterior fragmento de código y péguelo en Python, luego obtenga una captura de pantalla del resultado y péguelo aquí:

< ¡Resultado del fragmento de código aquí! >

Podemos resaltar los valores máximos y mínimos con colores.

```
format_dict = {'Mes': '{:%m-%Y}'} #Simplified format dictionary with
values that do make sense for our data

df.head().style.format(format_dict).highlight_max(color='darkgreen').h
ighlight_min(color='#ff0000')
```

Copie el anterior fragmento de código y péguelo en Python, luego obtenga una captura de pantalla del resultado y péguelo aquí:

< ¡Resultado del fragmento de código aquí! >

Utilizamos un gradiente de color para mostrar los valores de los datos.

```
df.head(10).style.format(format_dict).background_gradient(subset=['dat
a science', 'machine learning'], cmap='BuGn')
```

Copie el anterior fragmento de código y péguelo en Python, luego obtenga una captura de pantalla del resultado y péguelo aquí:

< ¡Resultado del fragmento de código aquí! >

También podemos mostrar los valores de los datos con barras.

```
df.head().style.format(format_dict).bar(color='red', subset=['data
science', 'deep learning'])
```

Copie el anterior fragmento de código y péguelo en Python, luego obtenga una captura de pantalla del resultado y péguelo aquí:

< ¡Resultado del fragmento de código aquí! >

Además, también podemos combinar las funciones anteriores y generar una visualización más compleja.

```
df.head(10).style.format(format_dict).background_gradient(subset=['dat
a science', 'machine learning'],
cmap='BuGn').highlight_max(color='yellow')
```

Copie el anterior fragmento de código y péguelo en Python, luego obtenga una captura de pantalla del resultado y péguelo aquí:

< ¡Resultado del fragmento de código aquí! >

Matplotlib

Matplotlib es la biblioteca más básica para visualizar datos gráficamente. Incluye muchos de los gráficos que se nos ocurren. Que sea básica no significa que no sea potente, muchas de las otras librerías de visualización de datos de las que vamos a hablar están basadas en ella.

Los gráficos de Matplotlib están formados por dos componentes principales, los ejes (las líneas que delimitan el área del gráfico) y la figura (donde dibujamos los ejes, títulos y cosas que salen del área de los ejes). Ahora vamos a crear el gráfico más sencillo posible:

```
import matplotlib.pyplot as plt
plt.plot(df['Mes'], df['data science'], label='data science') #The
parameter label is to indicate the legend. This doesn't mean that it
will be shown, we'll have to use another command that I'll explain
later.
```

Copie el anterior fragmento de código y péguelo en Python, luego obtenga una captura de pantalla del resultado y péguelo aquí:

< ¡Resultado del fragmento de código aquí! >

Podemos hacer las gráficas de múltiples variables en el mismo gráfico y así compararlas.

```
plt.plot(df['Mes'], df['data science'], label='data science')
plt.plot(df['Mes'], df['machine learning'], label='machine learning')
plt.plot(df['Mes'], df['deep learning'], label='deep learning')
```

Copie el anterior fragmento de código y péguelo en Python, luego obtenga una captura de pantalla del resultado y péguelo aquí:

< ¡Resultado del fragmento de código aquí! >

No está muy claro qué variable representa cada color. Vamos a mejorar el gráfico añadiendo una leyenda y títulos.

```
plt.plot(df['Mes'], df['data science'], label='data science')
plt.plot(df['Mes'], df['machine learning'], label='machine learning')
plt.plot(df['Mes'], df['deep learning'], label='deep learning')
plt.xlabel('Date')
plt.ylabel('Popularity')
plt.title('Popularity of AI terms by date')
plt.grid(True)
plt.legend()
```

Copie el anterior fragmento de código y péguelo en Python, luego obtenga una captura de pantalla del resultado y péguelo aquí:

< ¡Resultado del fragmento de código aquí! >

Si estás trabajando con Python desde la terminal o un script, después de definir el gráfico con las funciones que hemos escrito anteriormente utiliza `plt.show()`. Si estás trabajando desde Jupyter notebook, añade `%matplotlib inline` al principio del archivo y ejecútalo antes de hacer el gráfico.

Podemos hacer varios gráficos en una sola figura. Esto va muy bien para comparar gráficos o para compartir datos de varios tipos de gráficos fácilmente con una sola imagen.

```
fig, axes = plt.subplots(2,2)
axes[0, 0].hist(df['data science'])
axes[0, 1].scatter(df['Mes'], df['data science'])
axes[1, 0].plot(df['Mes'], df['machine learning'])
axes[1, 1].plot(df['Mes'], df['deep learning'])
```

Copie el anterior fragmento de código y péguelo en Python, luego obtenga una captura de pantalla del resultado y péguelo aquí:

< ¡Resultado del fragmento de código aquí! >

Podemos dibujar el gráfico con diferentes estilos para los puntos de cada variable:

```
plt.plot(df['Mes'], df['data science'], 'r-')
plt.plot(df['Mes'], df['data science']*2, 'bs')
plt.plot(df['Mes'], df['data science']*3, 'g^')
```

Copie el anterior fragmento de código y péguelo en Python, luego obtenga una captura de pantalla del resultado y péguelo aquí:

< ¡Resultado del fragmento de código aquí! >

Veamos ahora algunos ejemplos de los diferentes gráficos que podemos hacer con Matplotlib. Comenzamos con un gráfico de dispersión:

```
plt.scatter(df['data science'], df['machine learning'])
```

Copie el anterior fragmento de código y péguelo en Python, luego obtenga una captura de pantalla del resultado y péguelo aquí:

< ¡Resultado del fragmento de código aquí! >

Ejemplo de gráfico de barras:

```
plt.bar(df['Mes'], df['machine learning'], width=20)
```

Copie el anterior fragmento de código y péguelo en Python, luego obtenga una captura de pantalla del resultado y péguelo aquí:

< ¡Resultado del fragmento de código aquí! >

Ejemplo de histograma:

```
plt.hist(df['deep learning'], bins=15)
```

Copie el anterior fragmento de código y péguelo en Python, luego obtenga una captura de pantalla del resultado y péguelo aquí:

< ¡Resultado del fragmento de código aquí! >

Podemos añadir un texto al gráfico, indicamos la posición del texto en las mismas unidades que vemos en el gráfico. En el texto, podemos incluso añadir caracteres especiales siguiendo el lenguaje TeX. También podemos añadir marcadores que señalen un punto concreto del gráfico.

```
plt.plot(df['Mes'], df['data science'], label='data science')
plt.plot(df['Mes'], df['machine learning'], label='machine learning')
plt.plot(df['Mes'], df['deep learning'], label='deep learning')

plt.xlabel('Date')
plt.ylabel('Popularity')
plt.title('Popularity of AI terms by date')
plt.grid(True)

# Coordenadas x e y deben ser números que representen las coordenadas
# en el gráfico
plt.text(0.5, 80, r'$\lambda=1, r^2=0.8$')
```

Copie el anterior fragmento de código y péguelo en Python, luego obtenga una captura de pantalla del resultado y péguelo aquí:

< ¡Resultado del fragmento de código aquí! >

Galería de ejemplos:

En este enlace: <https://matplotlib.org/gallery/index.html> podemos ver ejemplos de todo tipo de gráficos que se pueden hacer con Matplotlib.

Seaborn

Seaborn es una biblioteca basada en Matplotlib. Básicamente, lo que nos da son gráficos más bonitos y funciones para hacer tipos de gráficos complejos con una sola línea de código.

Importamos la librería e inicializamos el estilo de los gráficos con `sns.set()`, sin este comando los gráficos seguirían teniendo el mismo estilo que Matplotlib. Mostramos uno de los gráficos más sencillos, un scatterplot.

```
import seaborn as sns
import pandas as pd
import matplotlib.pyplot as plt

# Se asume que el DataFrame 'df' ya está definido

sns.set()
sns.scatterplot(x='Mes', y='data science', data=df)
plt.show()
```

Copie el anterior fragmento de código y péguelo en Python, luego obtenga una captura de pantalla del resultado y péguelo aquí:

< ¡Resultado del fragmento de código aquí! >

Podemos añadir información de más de dos variables en el mismo gráfico. Para ello utilizamos colores y tamaños. También hacemos un gráfico diferente según el valor de la columna de la categoría:

```
sns.relplot(x='Mes', y='deep learning', hue='data science',  
size='machine learning', col='categorical', data=df)
```

Copie el anterior fragmento de código y péguelo en Python, luego obtenga una captura de pantalla del resultado y péguelo aquí:

< ¡Resultado del fragmento de código aquí! >

Uno de los gráficos más populares que proporciona Seaborn es el mapa de calor. Es muy común utilizarlo para mostrar todas las correlaciones entre las variables de un conjunto de datos:

```
sns.heatmap(df.corr(), annot=True, fmt='.2f')
```

Copie el anterior fragmento de código y péguelo en Python, luego obtenga una captura de pantalla del resultado y péguelo aquí:

< ¡Resultado del fragmento de código aquí! >

Otra de las más populares es el pairplot que nos muestra las relaciones entre todas las variables. Ten cuidado con esta función si tienes un conjunto de datos grande, ya que debe mostrar todos los puntos de datos tantas veces como columnas haya, significa que, al aumentar la dimensionalidad de los datos, el tiempo de procesamiento aumenta exponencialmente.

```
sns.pairplot(df)
```

Copie el anterior fragmento de código y péguelo en Python, luego obtenga una captura de pantalla del resultado y péguelo aquí:

< ¡Resultado del fragmento de código aquí! >

Ahora hagamos el pairplot mostrando los gráficos segmentados según los valores de la variable categórica.

```
sns.pairplot(df, hue='categorical')
```

Copie el anterior fragmento de código y péguelo en Python, luego obtenga una captura de pantalla del resultado y péguelo aquí:

< ¡Resultado del fragmento de código aquí! >

Un gráfico muy informativo es el `jointplot` que nos permite ver un gráfico de dispersión junto con un histograma de las dos variables y ver cómo se distribuyen:

```
sns.jointplot(x='data science', y='machine learning', data=df)
```

Copie el anterior fragmento de código y péguelo en Python, luego obtenga una captura de pantalla del resultado y péguelo aquí:

< ¡Resultado del fragmento de código aquí! >

Otro gráfico interesante es el `ViolinPlot`:

```
sns.catplot(x='categorical', y='data science', kind='violin', data=df)
```

Copie el anterior fragmento de código y péguelo en Python, luego obtenga una captura de pantalla del resultado y péguelo aquí:

< ¡Resultado del fragmento de código aquí! >

Podemos crear múltiples gráficos en una sola imagen al igual que hicimos con `Matplotlib`:

```
fig, axes = plt.subplots(1, 2, sharey=True, figsize=(8, 4))
sns.scatterplot(x="Mes", y="deep learning", hue="categorical",
data=df, ax=axes[0])
axes[0].set_title('Deep Learning')
sns.scatterplot(x="Mes", y="machine learning", hue="categorical",
data=df, ax=axes[1])
axes[1].set_title('Machine Learning')
```

Copie el anterior fragmento de código y péguelo en Python, luego obtenga una captura de pantalla del resultado y péguelo aquí:

< ¡Resultado del fragmento de código aquí! >

Galería de ejemplos:

[En este enlace](#), podemos ver ejemplos de todo lo que se puede hacer con Seaborn.

Bokeh

Bokeh es una librería que permite generar gráficos interactivos. Podemos exportarlos a un documento HTML que podemos compartir con cualquiera que tenga un navegador web.

Es una librería muy útil cuando nos interesa buscar cosas en los gráficos y queremos poder hacer zoom y movernos por el gráfico. O cuando queremos compartirlos y dar la posibilidad de explorar los datos a otra persona.

Comenzamos importando la librería y definiendo el archivo en el que guardaremos el gráfico:

```
from bokeh.plotting import figure, output_file, save
output_file('data_science_popularity.html')
```

Dibujamos lo que queremos y lo guardamos en el archivo:

```
p = figure(title='data science', x_axis_label='Mes',
           y_axis_label='data science')
p.line(df['Mes'], df['data science'], legend='popularity',
       line_width=2)
save(p)
```

Copie el anterior fragmento de código y péguelo en Python, luego obtenga una captura de pantalla del resultado y péguelo aquí:

< ¡Resultado del fragmento de código aquí! >

Añadir varios gráficos a un mismo archivo:

```

from bokeh.plotting import figure, gridplot, output_file, save
import pandas as pd

# Se asume que el DataFrame 'df' ya está definido

output_file('multiple_graphs.html')

s1 = figure(width=250, height=250, title='data science')
s1.circle(df['Mes'], df['data science'], size=10, color='navy',
alpha=0.5)

s2 = figure(width=250, height=250, x_range=s1.x_range,
y_range=s1.y_range, title='machine learning')
s2.triangle(df['Mes'], df['machine learning'], size=10, color='red',
alpha=0.5)

s3 = figure(width=250, height=250, x_range=s1.x_range, title='deep
learning')
s3.square(df['Mes'], df['deep learning'], size=5, color='green',
alpha=0.5)

p = gridplot([[s1, s2, s3]])
save(p)

```

Copie el anterior fragmento de código y péguelo en Python, luego obtenga una captura de pantalla del resultado y péguelo aquí:

< ¡Resultado del fragmento de código aquí! >

Galería de ejemplos:

En [este enlace](#) puedes ver ejemplos de todo lo que se puede hacer con Bokeh.

Folium

Folium es una biblioteca que nos permite dibujar mapas, marcadores y también podemos dibujar nuestros datos en ellos. Folium nos permite elegir el proveedor de mapas, esto determina el estilo y la calidad del mapa. En este artículo, para simplificar, sólo vamos a ver [OpenStreetMap](#) como proveedor de mapas.

Trabajar con mapas es bastante complejo y merece su propio artículo. Aquí sólo vamos a ver lo básico y a dibujar un par de mapas con los datos que tenemos.

Empecemos por lo básico, dibujaremos un mapa sencillo sin nada en él.

```
import folium
m1 = folium.Map(location=[19.357670, -99.180518],
tiles='openstreetmap', zoom_start=18)
m1.save('map1.html')
```

Copie el anterior fragmento de código y péguelo en Python, luego obtenga una captura de pantalla del resultado y péguelo aquí:

< ¡Resultado del fragmento de código aquí! >

Podemos añadir marcadores al mapa:

```
m2 = folium.Map(location=[41.38, 2.17], tiles='openstreetmap',
zoom_start=16)

folium.Marker([41.38, 2.176], popup='<i>You can use whatever HTML code
you want</i>', tooltip='click here').add_to(m2)
folium.Marker([41.38, 2.174], popup='<b>You can use whatever HTML code
you want</b>', tooltip='dont click here').add_to(m2)

m2.save('map2.html')
```

Copie el anterior fragmento de código y péguelo en Python, luego obtenga una captura de pantalla del resultado y péguelo aquí:

< ¡Resultado del fragmento de código aquí! >

En el conjunto de datos presentado al principio, tenemos los nombres de los países y la popularidad de los términos de inteligencia artificial. Tras una rápida visualización se puede ver que hay países en los que falta uno de estos valores. Vamos a eliminar estos países para facilitar la tarea. A continuación, utilizaremos Geopandas para transformar los nombres de los países en coordenadas que podamos dibujar en el mapa.

```
%pip install geopandas
import geopandas as gpd
import pandas as pd
import numpy as np
from geopy.geocoders import Nominatim

geolocator = Nominatim(user_agent="my-application") # Especifica tu
propio valor de user_agent

df2 = pd.read_csv('mapa.csv')
df2.dropna(axis=0, inplace=True)

geocoded = df2['País'].apply(lambda x: geolocator.geocode(x)) #
Geocodifica cada país

df2['geometry'] = geocoded.apply(lambda location: location.point if
location else None)
df2['Latitude'] = df2['geometry'].apply(lambda point: point.latitude
if point else None)
df2['Longitude'] = df2['geometry'].apply(lambda point: point.longitude
if point else None)
df2
```

Copie el anterior fragmento de código y péguelo en Python, luego obtenga una captura de pantalla del resultado y péguelo aquí:

< ¡Resultado del fragmento de código aquí! >

Ahora que tenemos los datos codificados en latitud y longitud, vamos a representarlos en el mapa. Empezaremos con un BubbleMap en el que dibujaremos círculos sobre los países. Su tamaño dependerá de la popularidad del término y su color será rojo o verde dependiendo de si su popularidad está por encima de un valor o no.

```
m3 = folium.Map(location=[39.326234,-4.838065], tiles='openstreetmap',
zoom_start=3)

def color_producer(val):
    if val <= 50:
        return 'red'
    else:
        return 'green'

for i in range(0,len(df2)):
    folium.Circle(location=[df2.iloc[i]['Latitude'],
df2.iloc[i]['Longitude']], radius=5000*df2.iloc[i]['data science'],
color=color_producer(df2.iloc[i]['data science'])).add_to(m3)

m3.save('map3.html')
```

Copie el anterior fragmento de código y péguelo en Python, luego obtenga una captura de pantalla del resultado y péguelo aquí:

< ¡Resultado del fragmento de código aquí! >

¿Qué biblioteca utilizar en cada momento?

Con toda esta variedad de librerías te preguntará qué librería es la mejor para tu proyecto. La respuesta rápida es la librería que te permita hacer fácilmente el gráfico que quieras.

Para las fases iniciales de un proyecto, con Pandas haremos una visualización rápida para entender los datos. Si necesitamos visualizar más información, podemos utilizar gráficos sencillos que podemos encontrar en matplotlib como scatterplots o histogramas.

Para fases avanzadas del proyecto, podemos buscar en las galerías de las principales librerías (Matplotlib, Seaborn, Bokeh, Altair) los gráficos que nos gusten y se ajusten al proyecto. Estos gráficos se pueden utilizar para dar información en informes, hacer informes interactivos, buscar valores concretos, etc.